Mouse RNA-Seq — QC Remediation + Full `fastp` Validation

BIOL550 — Weekly Report — Mar 11, 2026

What I accomplished since the previous report

This week, in communication with my team, I explored the QC remediation and validation phase for our mouse bulk RNA-seq dataset (`PRJNA1017789`; `GSE243308`). This includes explicit cleanup for the residual adapter-related signal, re-running FastQC after cleanup, generating a higher-level QC summary, and moving closer to the alignment stage. I compared the `FASTX` baseline, `FASTX + cutadapt`, and `fastp` on the most problematic pilot reads first and then across all **52** read-level reports, which shows `fastp` to be the right tool for cleanup. Then, I ran `fastp` across all **26 paired-end SRRs**, reran FastQC on all **52** cleaned read-level reports, and generated MultiQC reports for the FASTX baseline and the full FASTX-vs-`fastp` comparison.

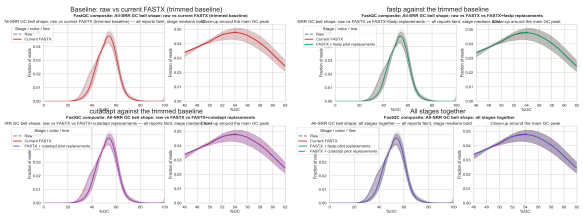
After learning the cleanup tools more deeply, I used them to explore how the data behaves before and after each remediation strategy to better inform tool selection while building a defensible, evidence-based path as we prepare for alignment.

Following previous file-by-file QC approach, I parsed and compared FastQC outputs directly for each read report across stages to validate MultiQC results. Using a notebook, I noted the findings of my research and analysis for the team to discuss next week.

Full-dataset QC outcome summary

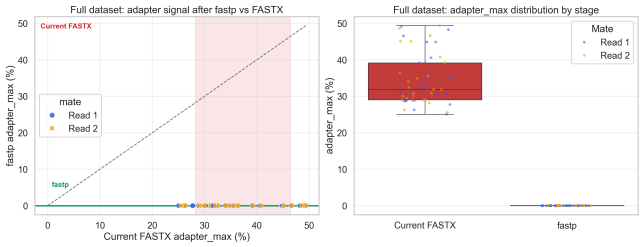
Metric	FASTX baseline	After full `fastp` rerun	Interpretation
Reports compared	52 read-level FastQC reports	52 read-level FastQC reports	Full-dataset comparison, not only pilot reads.
Median `adapter_max`	31.8806	0.0051	Residual adapter signal collapsed to near zero after `fastp`.
Adapter Content	0 PASS / 0 WARN / 52 FAIL	52 PASS / 0 WARN / 0 FAIL	Strongest categorical cleanup effect.
Overrepresented sequences	37 PASS / 13 WARN / 2 FAIL	52 PASS / 0 WARN / 0 FAIL	Technical overrepresentation signal also cleared completely.
Median retained reads	—	97.52%	Cleanup benefit did not come from aggressive read loss.
Median post-`fastp` Q30	—	93.83%	Post-cleanup read quality remained strong.

Dataset-level stage overview before the file-by-file drill-down



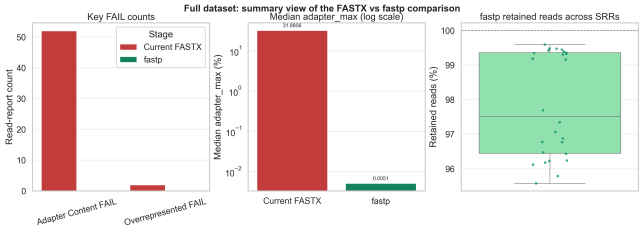
The image to the left is the all-SRR stage-level view across the raw baseline, the current FASTX-trimmed baseline, and the pilot replacement views for `fastp` and `cutadapt`. At this aggregate scale the curves look very similar, which means the overall library shape remains stable across cleanup strategies. This is why we the file-by-file comparison that follows is important: the meaningful remediation differences exist, but they do not stand out clearly in the dataset-level summary alone.

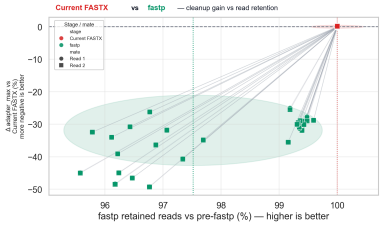
Tool-to-tool comparison: how `fastp` differs from the trimmed FASTX baseline



The figures above show how the full-dataset FASTX baseline differs from the `fastp`-cleaned stage. The direct FASTX-vs-`fastp` comparison shows that the residual adapter signal drops sharply after `fastp`, while the status matrix shows the same change categorically: `Adapter Content` and `Overrepresented sequences` move out of FASTX FAIL/WARN states and into `fastp` PASS states. Read together, they show that the two stages are not similar views of the same data; they behave differently at the continuous-signal level and the module-status level.

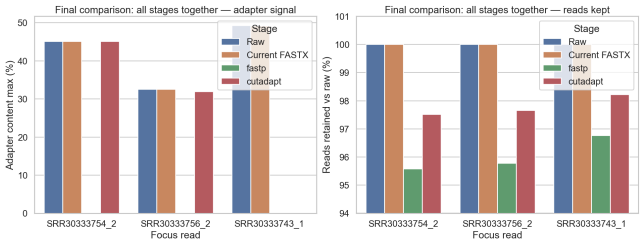
Why `fastp` is better: categorical QC cleanup (left) + cleanup gain vs read retention (right)

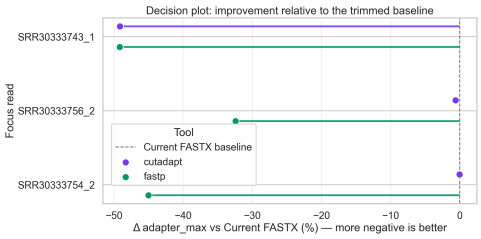




The figures above summarize why `fastp` is the better cleanup stage at the full-dataset scale. The summary view shows that the FASTX baseline retains the key FAIL counts and a high median `adapter_max`, whereas `fastp` collapses that signal and preserves strong read retention. The retention plot then makes the tradeoff explicit: the cleanup gain is large, but the retained-read percentages still stay around `95.5–99.5%`, so the improvement does not come from major read loss.

Focused pilot-read view: the narrowest comparison used to close the decision





At the narrowest level, the pilot-read comparison (seen above) shows the same conclusion as the full-dataset analysis. `cutadapt` retains more reads than `fastp` on the focus reads, but the improvement plot shows that `fastp` removes much more residual adapter signal relative to the trimmed FASTX baseline, clearly showing why `fastp` is a better cleanup tool for this dataset.

Methods used (commands + parameters)

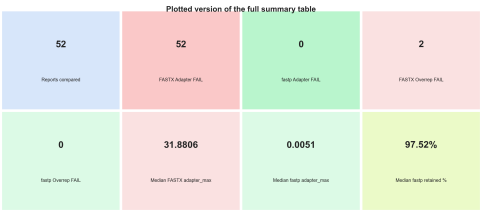
We kept the same reproducible workflow structure as before, but this week the main cleanup method changed from the older FASTX quality-only trim to a validated `fastp`-based cleanup stage. For the full rerun, I processed all 26 paired-end SRRs with `fastp` using paired-end adapter detection, quality-tail trimming, minimum-length enforcement, and poly-G cleanup. Then, I reran FastQC on all cleaned files, compared the resulting FastQC outputs directly against the FASTX baseline at the file/module level, and generated supplementary MultiQC reports for integrated review.

```
fastp \
  --detect_adapter_for_pe \
  --cut_tail \
  --cut_mean_quality 20 \
  --length_required 30 \
  --trim_poly_g

Validation layers used this week:
1) custom file-by-file FastQC comparison (primary evidence)
2) FASTX baseline MultiQC + FASTX-vs-fastp MultiQC (supplementary
confirmation)
```

The same QC comparison approach as last week was used to parse the outputs produced by the remediation strategies (`FASTX`, `cutadapt`, and `fastp`). The image on the right shows how each tool performs on the problematic reads by comparing reads retained (%) against residual adapter signal, demonstrating that `fastp` performed best overall among the tested tools.

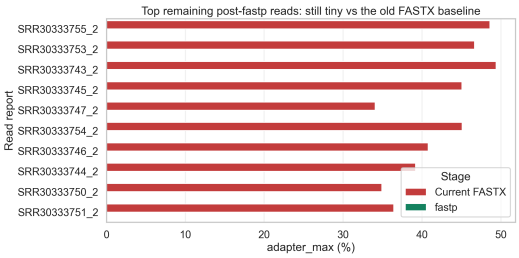
Problems encountered



As seen in the validation summary output on the left, the original FASTX-trimmed baseline was still not alignment-ready for the technical artifact modules we were trying to fix. The summary shows `52 Adapter Content` FAIL reports, `2 Overrepresented sequences` FAIL reports, and a median `adapter_max` of `31.8806` under FASTX. This is the evidence that the residual adapter-related signal was still present after the first trimming strategy.

The same validation summary also shows what changed after `fastp` and what did not. `Adapter Content` FAIL dropped from `52` to `0`, `Overrepresented sequences` FAIL dropped from `2` to `0`, and median `adapter_max` dropped from `31.8806` to `0.0051`. However, not every FastQC flag changed after `fastp`: `Per base sequence content` remained split across `26 WARN` and `26 FAIL`, and `Sequence Length Distribution` remained `52 WARN`. Based on the file-by-file review and the course guidance, we might treat these as signals to monitor rather than reasons to keep trimming, because they are consistent with RNA-seq composition effects and expected post-trim read-length variability.

Goals for the coming week

Next week, in collaboration with my team, I will use these findings to help speed up alignment preparation after the team reviews and validates them. As shown in the remaining post-`fastp` reads summary on the right, even the highest residual reads are still tiny relative to the old FASTX baseline, so the immediate next step would be to confirm the alignment inputs.

Similarly, I will (1) present the notebook-based alignment-prep discussion items to the team and confirm the cleaned `fastp` inputs are ready for alignment and document why `cutadapt` is not needed as a fallback, (2) confirm the exact mouse reference genome + annotation pair for alignment, (3) build the sample/design sheet with SRR, condition, replicate, and mate-path columns, and (4) prepare the STAR run manifest and the alignment-QC capture template so we can start mapping with the cleaned `fastp` outputs rather than the older FASTX baseline.